CSE471: System Analysis and Design
Project Report
Project Title: AcademiX

Group No: 3 CSE471 Lab Section: 9 Fall 2024	
ID	**Name**
24141253	Ahmed Riasat
22101215	Mustafina Joarder

Submission Date: 13/05/2025

Table of Contents

1. System Request 3
   Business need: 3
   Business requirements: 3
   Business value: 3
   Special issues or constraints: 3
2. Functional Requirements 4
3. Technology (Framework, Languages) 4
4. Backend Development 5
5. User Interface Design 11
6. Frontend Development 14
7. User Manual 18
8. Performance and Network Analysis 24
9. Github Repo [Public] Link 27
10. Link of Deployed Project 28
11. Individual Contribution 28
12. References 28

1. System Request

Business need: Colleges and universities increasingly require digital tools to streamline academic activities. Managing quizzes, assignments, and tracking student progress regardless if it is conducted online or offline - remains a major challenge without a centralized platform. Here we propose "AcademiX" as a solution for this problem.

Business requirements: These are the features that the system must provide:

1. Account management:
    a. Account creation based on role (Admin, Teacher, Student)
    b. Profile and password management
    c. Admin control over other users

2. Dashboard
    a. Custom dashboard based on role
    b. Role based view and access from dashboard

3. Course management
    a. Admin can create courses and assign teachers in that course
    b. Amin and teacher can create and control assignment
    c. Student can enroll in courses and attend assignment and quizzes
    d. Score tracking

Business value: This system will be beneficial for both teacher and students. It will be a centralized platform for keeping track of assignments and quizzes. There will be no need to visit several platforms for different courses. Everything will be under one platform. There will be no last moment searching of quiz questions and assignment questions before exams. It will support both online and offline works.

Special issues or constraints:

1. Role based access and view
2. Offline quizzes can also be handles
3. Score tracking
4. Submission of pdf and docs

2. Functional Requirements

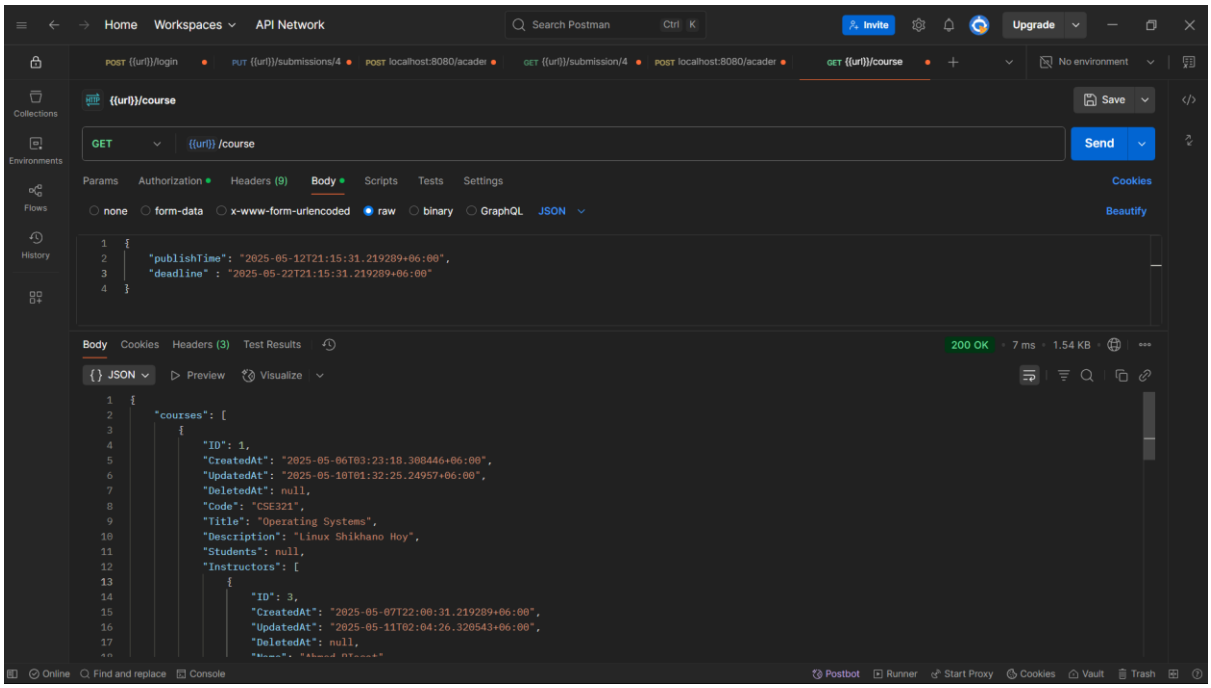
- Users can register with a username, valid email, password and role
- User can do successful login logout
- Password and profile can be updated
- Admin can control other user's profile
- Admin can create course and assign teacher
- Admin and Teacher can create assignments and quizzes
- Students can enroll in courses
- Score can be tracked
- Student can view their submitted works

3. Technology (Framework, Languages)

- Backend Language - Golang
- Backend framework - Gin
- Frontend Language - Typescript
- Frontend Library - React
- Frontend Framework - Next js
- Database - PostgreSQL

4. Backend Development

1. academix/course ->this route fetches all the course available

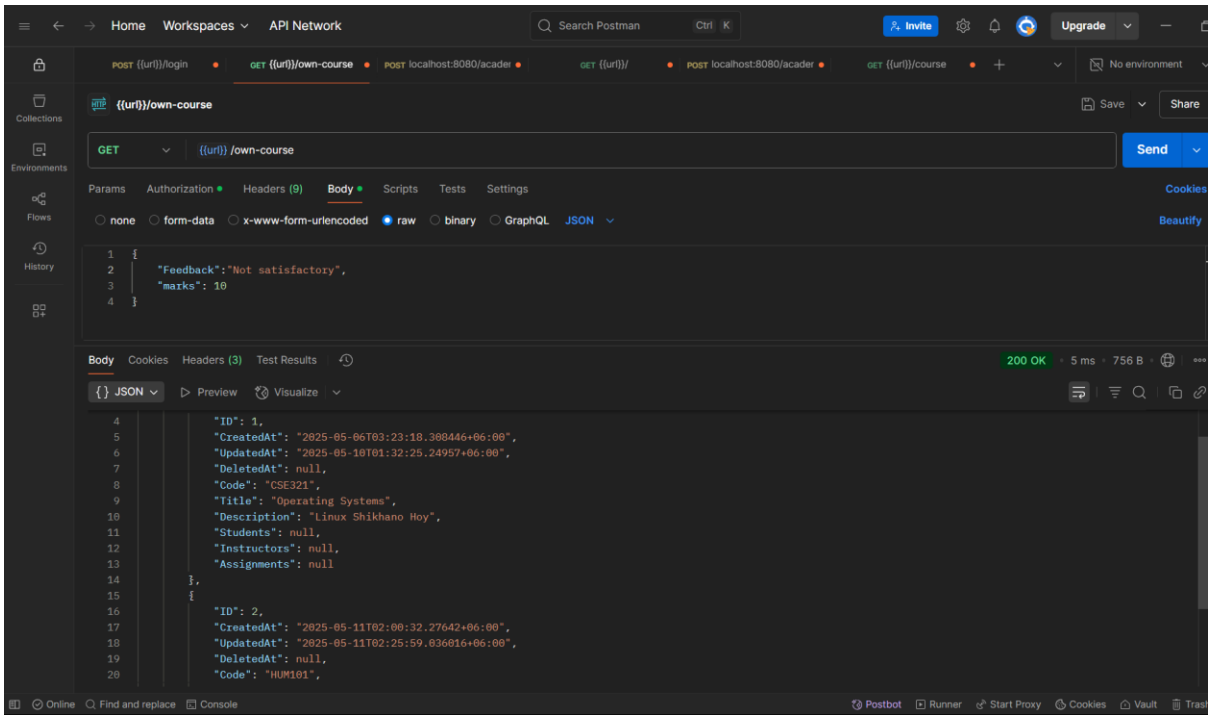



```go
func ViewAllCourses(c *gin.Context) { //view all courses  1 usage  Ahmed Riasat

    role := c.GetString( key: "role")

    if !permissions.ValidatePermission(role,  resource: "course",  expectedPermission: "viewAll") {
        c.JSON(http.StatusUnauthorized, gin.H{"error": "Invalid Permission"})
        return
    }

    var courses []models.CourseModel

    if err := database.DB.Preload( query: "Instructors").Find(&courses).Error; err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error(), "message": "Error Fetching Data"})
        return
    }
    c.JSON(http.StatusOK, gin.H{"courses": courses})
}
```

This composite literal allocates a new map instance with the given values.

2.academix/own-course -> this route fetches the courses a student is enrolled in or a teacher teaching the course (in this screenshot it's teacher's view)

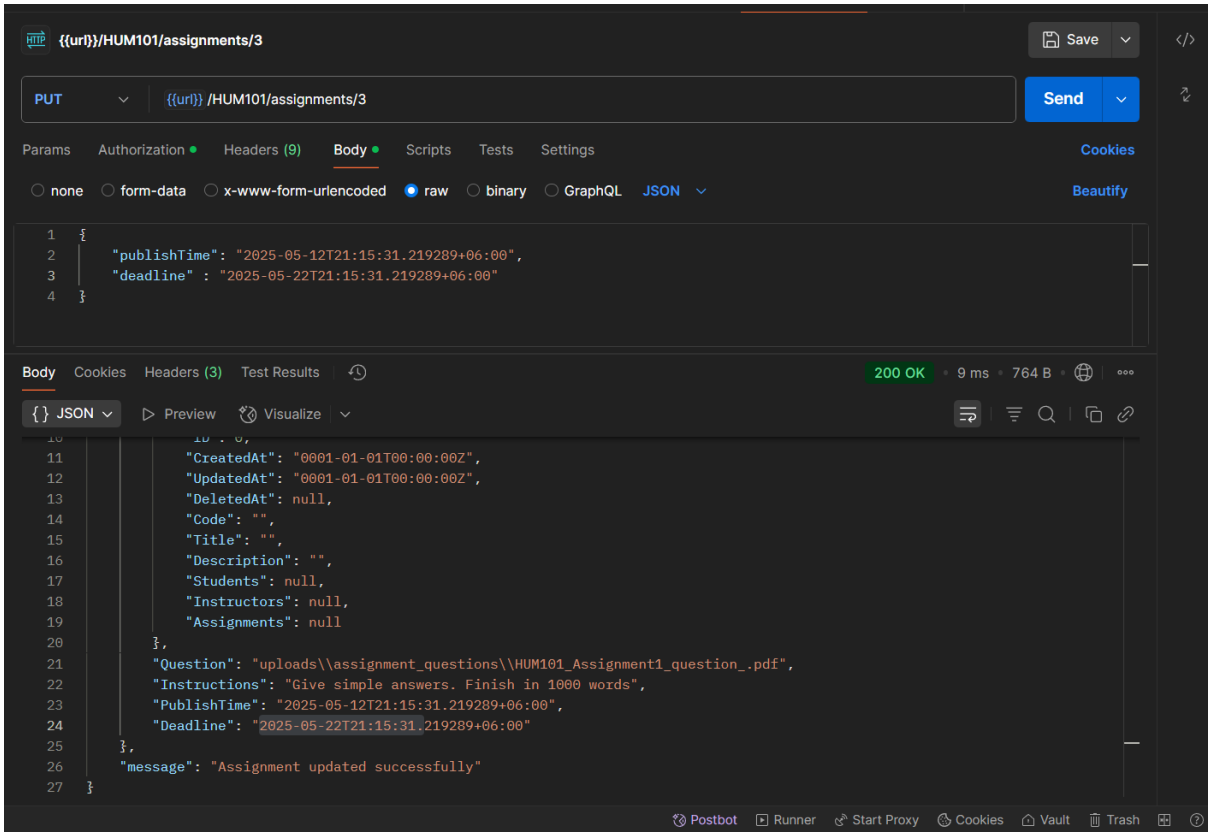
```go
func ViewOwnCourses(c *gin.Context) {  1 usage  Ahmed Riasat
    role := c.GetString( key: "role")
    username := c.GetString( key: "username")

    if !permissions.ValidatePermission(role,  resource: "course",  expectedPermission: "viewOwn") {
        c.JSON(http.StatusUnauthorized, gin.H{"error": "Invalid Permission"})
        return
    }
    var courses []models.CourseModel
    /*  alternative
        database.DB.Where("username = ?", user.Username).Find(&courses)
    */
    if role == "student" {
        courses = GetStudentCourses(username)
    }
    if role == "teacher" {
        courses = GetInstructorCourses(username)
    }

    c.JSON(http.StatusOK, gin.H{"courses": courses})
}
```



3. /:courseCode/assignments/:assignment_id -> here we get an particular assignment(in this view it was being updated)

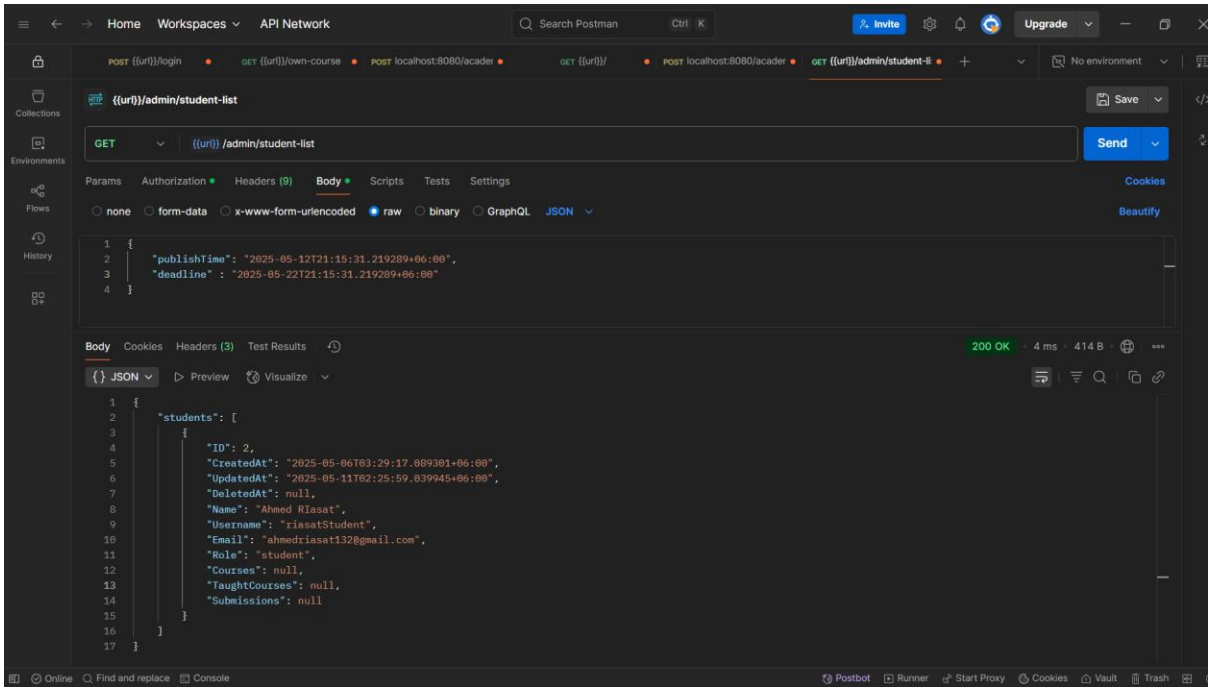
```go
func UpdateAssignment(c *gin.Context) {  1 usage  Ahmed Riasat
    if !permissions.ValidatePermission(role, resource: "assignment", expectedPermission: "edit") {
        c.JSON(http.StatusUnauthorized, gin.H{"error": "Invalid Permission"})
        return
    }

    assignmentIDStr := c.Param( key: "assignment_id")
    assignmentID, err := strconv.ParseUint(assignmentIDStr, base: 10, bitSize: 32)
    if err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": "Invalid assignment ID"})
        return
    }

    var assignment models.Assignment
    if err := database.DB.Where( query: "id = ?", assignmentID).First(&assignment).Error; err != nil {
        c.JSON(http.StatusNotFound, gin.H{"error": "Assignment not found"})
        return
    }

    var input struct {
        Serial      *int       `json:"serial"`      // Pointer so that if omitted, we don't update it.
        Instruction *string    `json:"instruction"` // Pointer so that we can accept null or missing values.
        PublishTime *time.Time `json:"publishTime"` // Pointer to update only when provided.
        Deadline    *time.Time `json:"deadline"`
    }

    if err := c.ShouldBindJSON(&input); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": "Invalid input format"})
        return
    }

    if input.Serial != nil {
        assignment.Serial = *input.Serial
    }
    if input.Instruction != nil {
        assignment.Instructions = input.Instruction
    }
    if input.PublishTime != nil {
```

```go
func UpdateAssignment(c *gin.Context) {  1 usage  Ahmed Riasat
        // Validate file extension.
        ext := strings.ToLower(filepath.Ext(questionFile.Filename))
        if ext != ".pdf" && ext != ".doc" && ext != ".docx" {
            c.JSON(http.StatusBadRequest, gin.H{"error": "Unsupported file format for question"})
            return
        }

        uploadDir := "uploads/assignment_questions"
        if _, err := os.Stat(uploadDir); os.IsNotExist(err) {
            if errDir := os.MkdirAll(uploadDir, perm: 0755); errDir != nil {
                c.JSON(http.StatusInternalServerError, gin.H{"error": "Could not create upload directory"})
                return
            }
        }

        filename := fmt.Sprintf( format: "%s_Assignment%d_question_%s", assignment.CourseCode, assignment.Serial, ext)
        filePath := filepath.Join(uploadDir, filename)

        if err := c.SaveUploadedFile(questionFile, filePath); err != nil {
            c.JSON(http.StatusInternalServerError, gin.H{"error": "Failed to save updated question file"})
            return
        }
        assignment.Question = filePath
    }

    if err := database.DB.Save(&assignment).Error; err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Failed to update assignment"})
        return
    }

    c.JSON(http.StatusOK, gin.H{
        "message":    "Assignment updated successfully",
        "assignment": assignment,
    })
    return
}
```

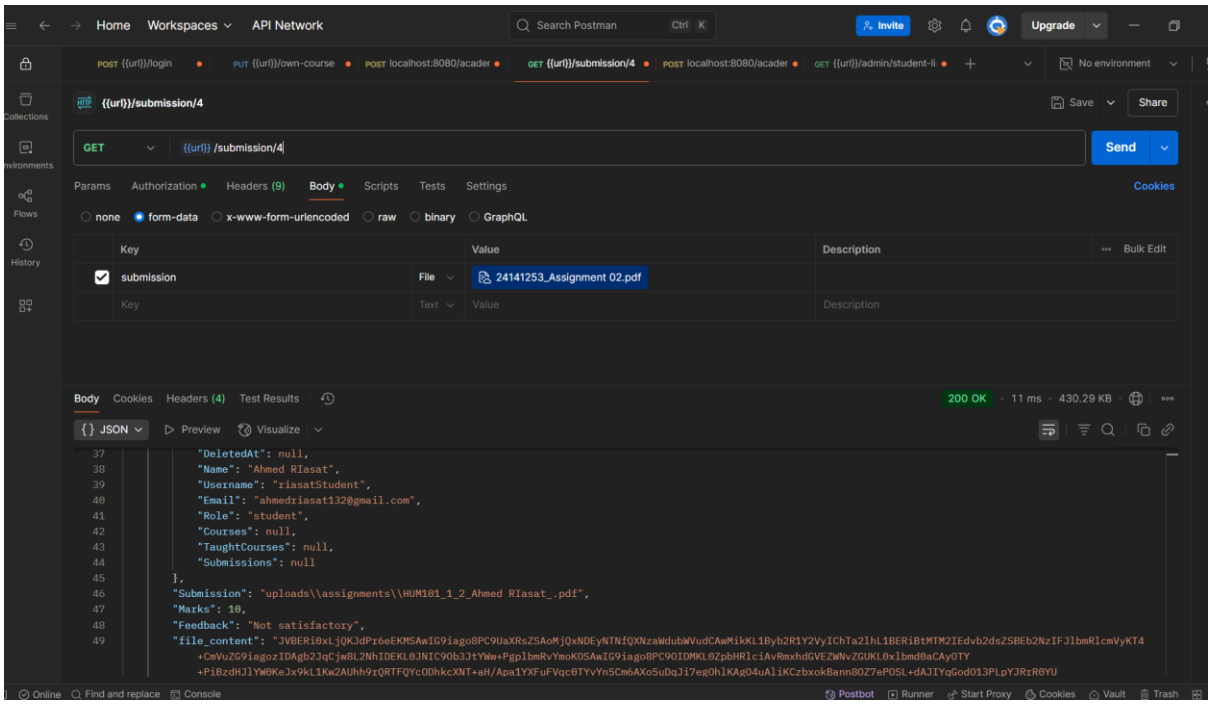


4./admin/student-list -> fetches the studentlist (only for admin view)

```go
func GetStudentList(c *gin.Context) {  1 usage  Ahmed Riasat
    role := c.GetString( key: "role")
    if !permissions.ValidatePermission(role,  resource: "user",  expectedPermission: "view") {
        c.JSON(http.StatusOK, gin.H{"message": "Permission Denied", "role": role})
    }
    var students []models.UserModel
    if err := database.DB.Where( query: "role = ?",  args...: "student").Find(&students).Error; err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Failed to fetch students"})
        return
    }
    c.JSON(http.StatusOK, gin.H{"students": students})

}
```



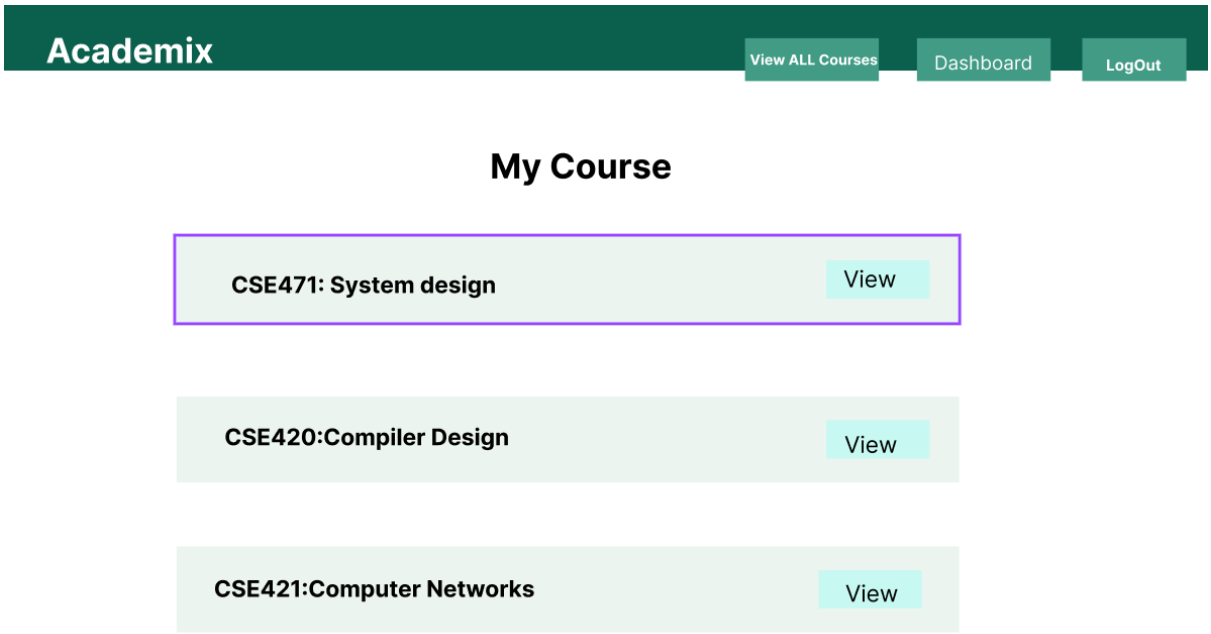
5./submissions/:submission_id -> student receives his received marks and feedback

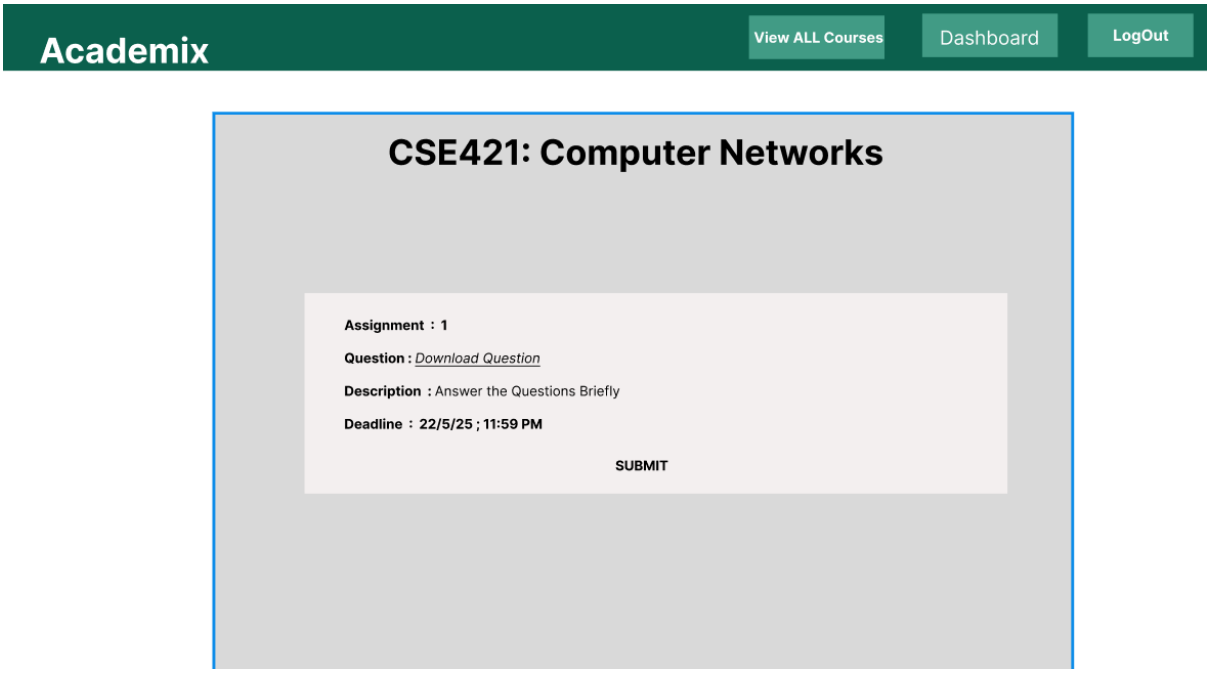


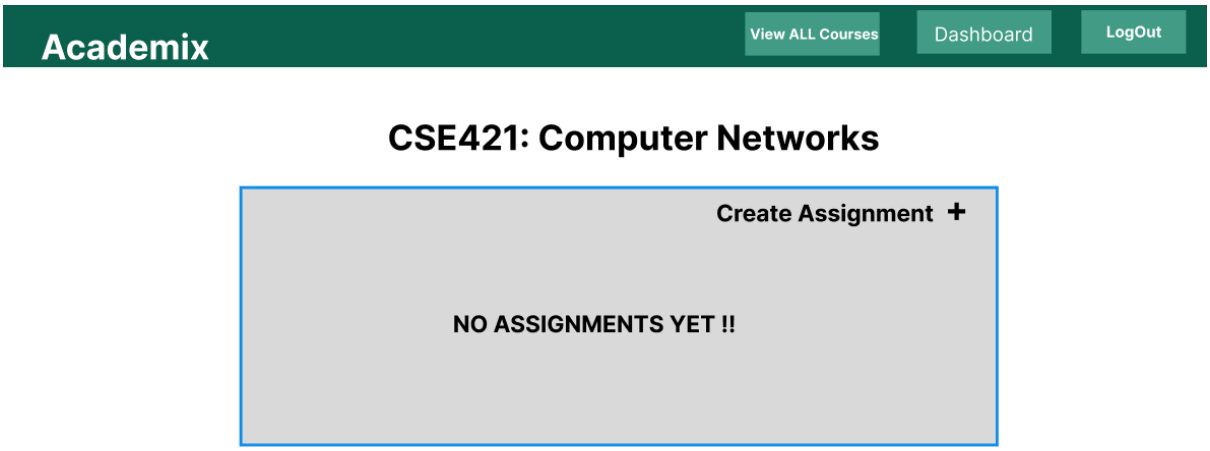
```go
func GetStudentSubmissions(c *gin.Context) {  1 usage  Ahmed Riasat

    role := c.GetString( key: "role")
    if !permissions.ValidatePermission(role,  resource: "submission",  expectedPermission: "getMarks:Feedback") {
        c.JSON(http.StatusUnauthorized, gin.H{"error": "Invalid Permission"})
        return
    }

    //username := c.GetString("username")
    submissionID, err := strconv.Atoi(c.Param( key: "submission_id"))
    if err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": "Invalid submission ID"})
    }
    var submission models.AssignmentSubmission
    if err := database.DB.Preload( query: "Student").
        Where( query: "id = ? ", submissionID).
        First(&submission).Error; err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Failed to fetch student data"})
        return
    }
    if submission.Submission != "" {
        content, err := os.ReadFile(submission.Submission)
        if err != nil {
            // Log or handle the error as needed; here we set empty content.
            submission.FileContent = ""
        } else {
            submission.FileContent = base64.StdEncoding.EncodeToString(content)
        }
    }

    c.JSON(http.StatusOK, gin.H{"submission": submission})
    return
}
```

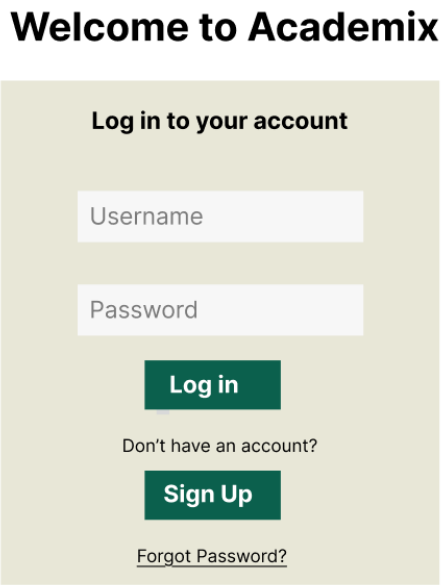
5. User Interface Design

ProjectLink : https://www.figma.com/design/ckABt25cTC58QMxoGrHTxG/Untitled?node-id=0-1&t=34yyaqH1avLgIdYX-1




Academix
View ALL Courses
Dashboard
LogOut
CSE421: Computer Networks
Assignment : 1
Question : Download Question
Description : Answer the Questions Briefly
Deadline : 22/5/25 ; 11:59 PM
SUBMIT


Academix
View ALL Courses
Dashboard
LogOut
CSE421: Computer Networks
Create Assignment +
NO ASSIGNMENTS YET !!



6. Frontend Development

1.Handling login with token

```
const handleLogin = async () => {
  try {
    const response = await fetch("http://localhost:8080/academix/login", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      credentials : "include",
      body: JSON.stringify({
        username: username.trim(), //trim removes accidental empty spaces
        password: password.trim()
      })
    });

    const result = await response.json();
    const token = result.token;

    if (response.ok && token) {
      localStorage.setItem("authToken", token);
      router.push("/dashboard");
    } else {
      alert(result.error || "Invalid username or password.");
    }
  } catch (error) {
    console.error("Error logging in:", error);
    alert("Something went wrong. Please try again.");
  }
};
```

2. Updating email address on profile page

```
const handleUpdateEmail = async (e: React.FormEvent) => {
  e.preventDefault();
  const token = localStorage.getItem("authToken");
  try {
    const response = await fetch("http://localhost:8080/academix/profile", {
      method: "PUT",
      headers: {
        "Content-Type": "application/json",
        Authorization: "Bearer " + token,
      },
      credentials: "include",
      body: JSON.stringify({ email: newEmail }),
    });
    const result = await response.json();
    if (response.ok) {
      alert("Email updated successfully!");
      setProfile({ ...profile!, Email: newEmail });
      setEditMode(false);
    } else {
      alert(result.error || "Update failed.");
    }
  } catch (error) {
    console.error("Error updating email:", error);
  }
};
```

3. Student can enroll in a course

```
// Handle enrolling in a course
const handleEnroll = async (courseCode: string) => {
  if (!confirm("Do you want to enroll in this course?")) return;
  try {
    const token = localStorage.getItem("authToken");
    const response = await fetch(`http://localhost:8080/academix/enroll-course/${courseCode}`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Authorization": "Bearer " + token,
      },
      credentials: "include",
    });
    const result = await response.json();
    if (response.ok) {
      alert("Enrolled successfully!");
    } else {
      alert(result.error || "Enrollment failed.");
    }
  } catch (error) {
    console.error("Error during enrollment:", error);
    alert("Something went wrong. Please try again.");
  }
};

if (loading) {
  return (
    <div style={{ fontFamily: "Arial, sans-serif", textAlign: "center", marginTop: "50px" }}>
      <h2>Loading courses...</h2>
    </div>
  );
}
```

4. Teacher can create an assignment

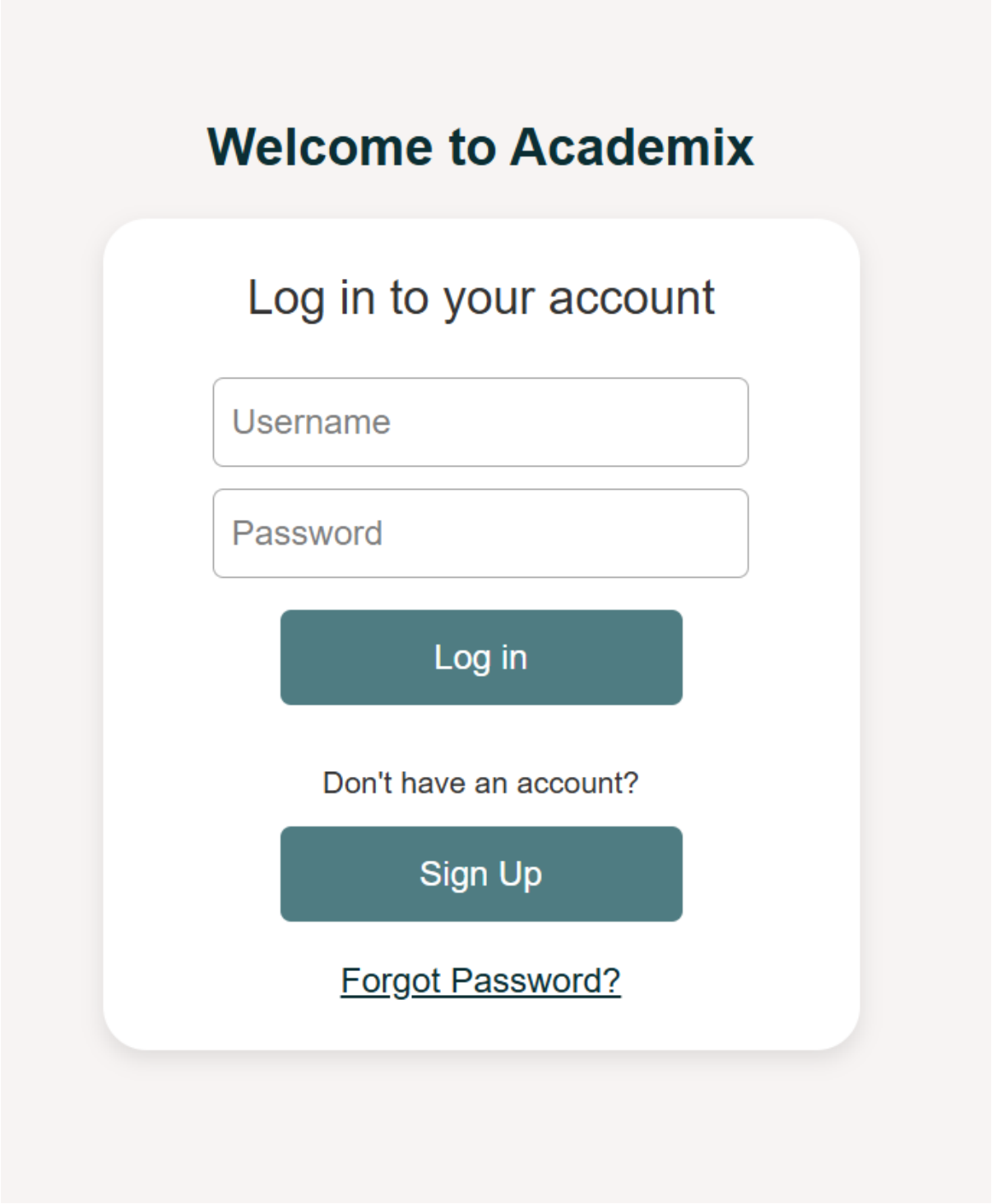
```
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  const token = localStorage.getItem("authToken");
  if (!questionFile) {
    alert("Please attach a question file.");
    return;
  }
  const formData = new FormData();
  formData.append("serial", serial.toString());
  formData.append("instruction", instruction);
  formData.append("publishTime", publishTime);
  formData.append("deadline", deadline);
  formData.append("question", questionFile);
  try {
    const response = await fetch(`http://localhost:8080/academix/${courseCode}/assignment`, {
      method: "POST",
      headers: { Authorization: "Bearer " + token },
      credentials: "include",
      body: formData,
    });
    const result = await response.json();
    if (response.ok) {
      alert("Assignment created successfully!");
      router.push(`/course/${courseCode}`);
    } else {
      alert(result.error || "Failed to create assignment.");
    }
  } catch (error) {
    console.error("Error creating assignment:", error);
  }
};
```

5. Viewing student list as an admin

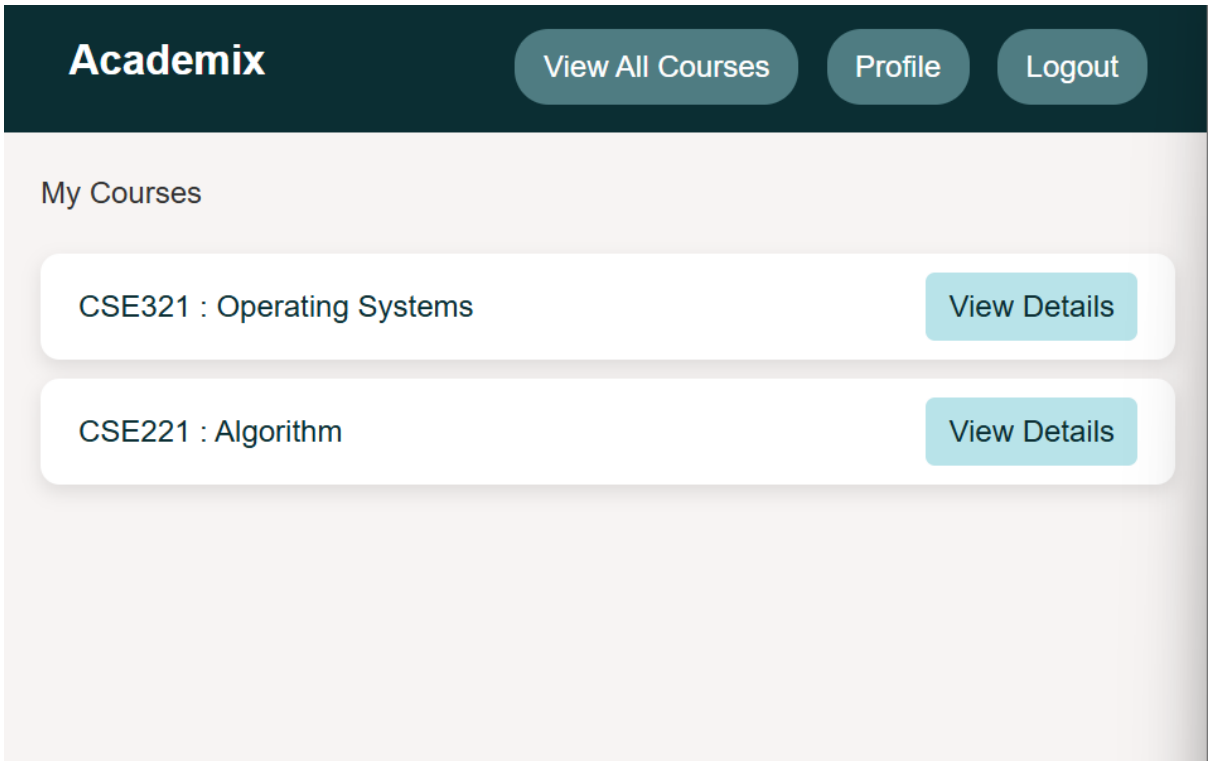
```
useEffect(() => {
  const token = localStorage.getItem("authToken");
  if (token) {
    fetch("http://localhost:8080/academix/admin/student-list", {
      method: "GET",
      headers: {
        "Content-Type": "application/json",
        "Authorization": "Bearer " + token,
      },
      credentials: "include",
    })
      .then((res) => res.json())
      .then((data) => {
        if (data.students) setStudents(data.students);
        else alert("Could not fetch student list.");
        setLoading(false);
      })
      .catch((error) => {
        console.error("Error fetching student list:", error);
        setLoading(false);
      });
  }
}, []);

if (!authorized) return null;
if (loading) {
  return <div style={{ textAlign: "center", marginTop: "50px", color: "#000" }}>Loading student list...</div>;
}

return (
  <div style={{ fontFamily: "Arial, sans-serif", minHeight: "100vh", backgroundColor: "#F7F4F3", padding: "20px", color: "#000" }}>
    <h2 style={{ textAlign: "center", marginBottom: "20px" }}>Student List</h2>
    {students.length > 0 ? (
      students.map((student) => (
        <div
          key={student.ID}
          style={{ backgroundColor: "#FFFFFF", padding: "15px", borderRadius: "10px", marginBottom: "15px", boxShadow: "0px 4px 10px rgba(0,0,0,0.1)", maxWidth: "600px", marg
        >
          <p><strong>Name:</strong> {student.Name}</p>
          <p><strong>Username:</strong> {student.Username}</p>
          <p><strong>Email:</strong> {student.Email}</p>
        </div>
```

7. User Manual

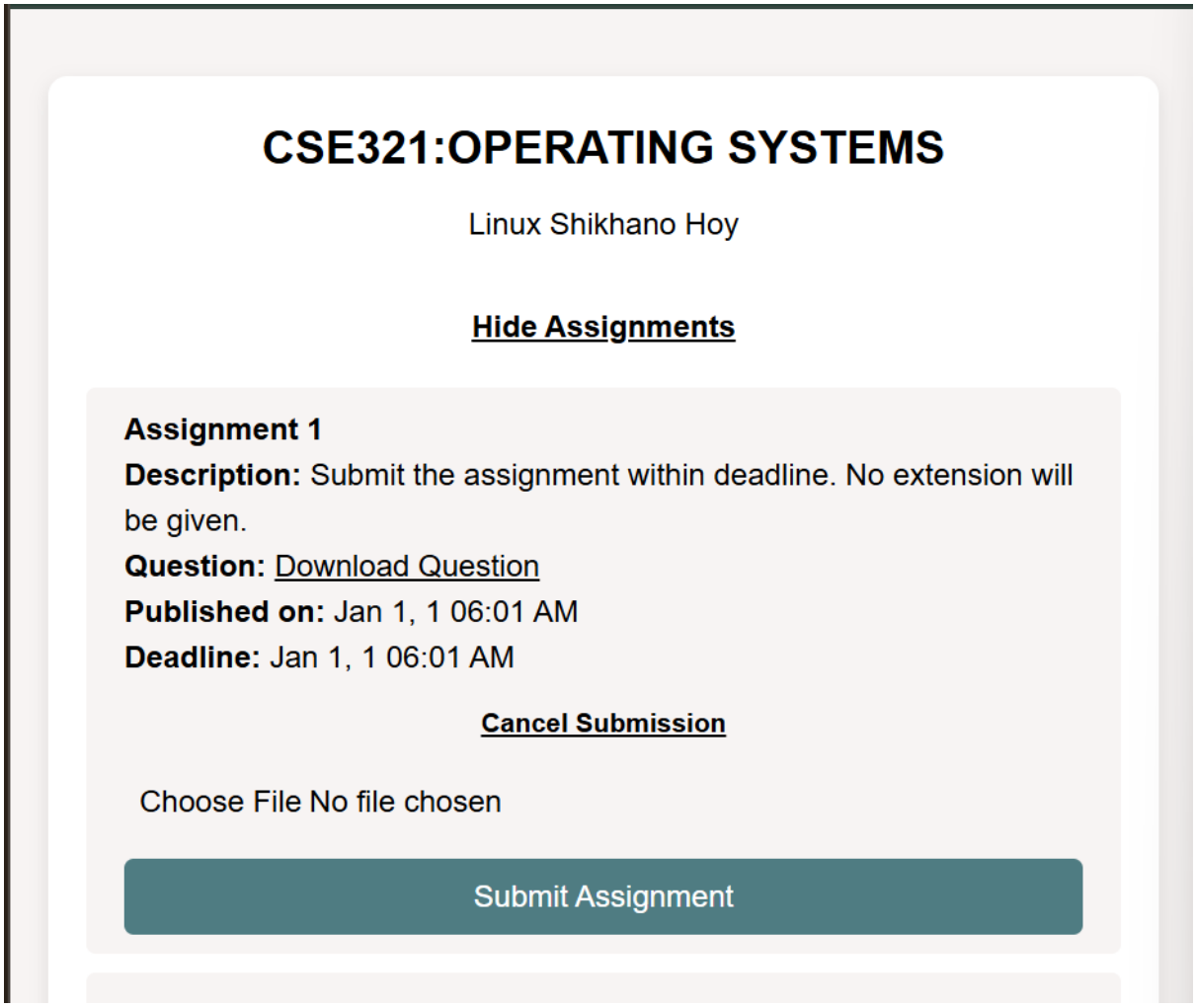
1. Landing page of the website



2. After successful login, the student will view this page where he can see the courses he is currently enrolled in. The “View All Courses” button shows the student courses created by admin and he can enroll in them from here.



3. Clicking “View Details” takes the student inside the course and he can see the assignments created by teacher and also can submit the assignment.



4. If the logged in user is a Teacher, he will see this page inside a course.

CSE321:OPERATING SYSTEMS

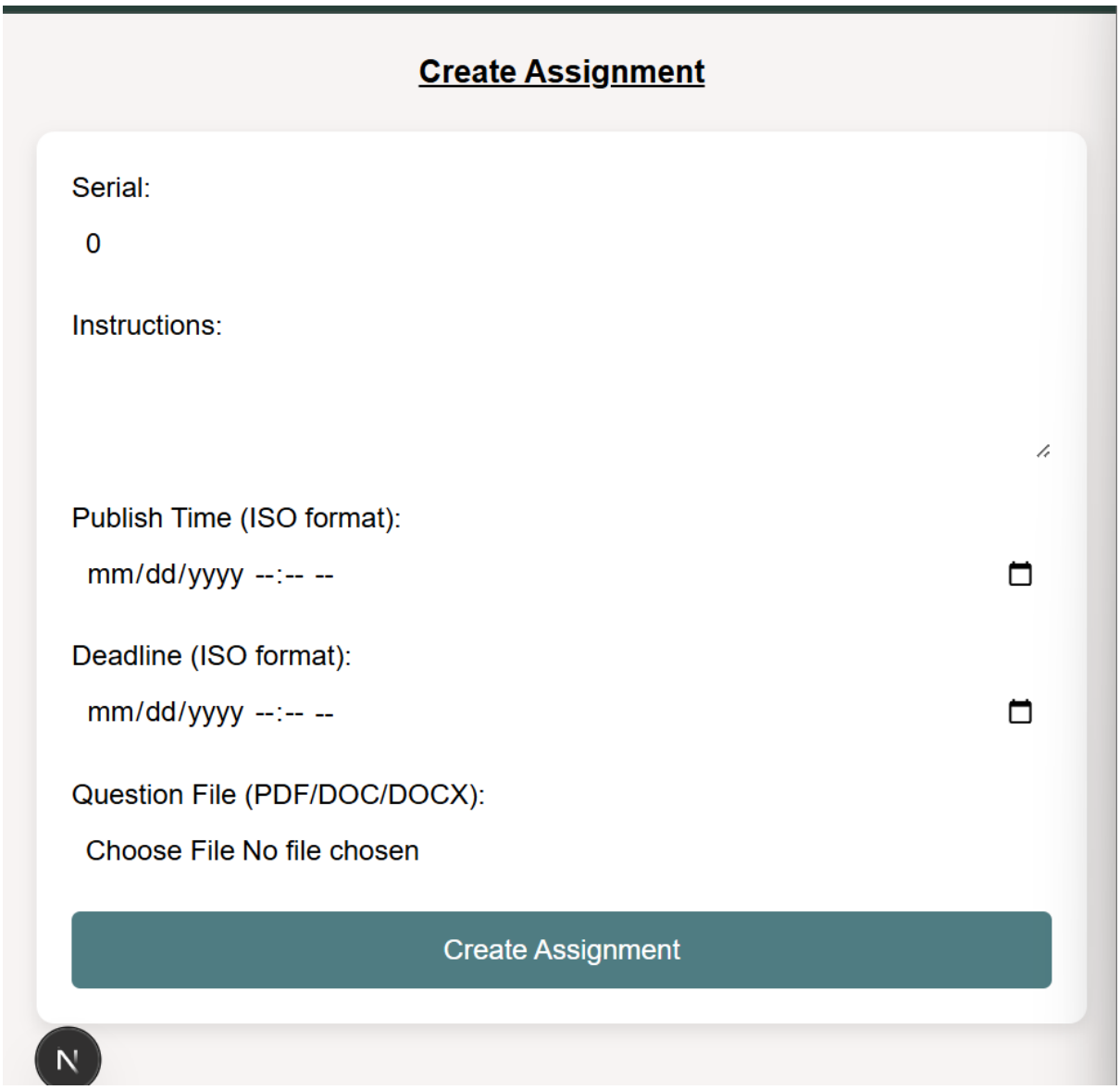
Linux Shikhano Hoy

Create Assignment

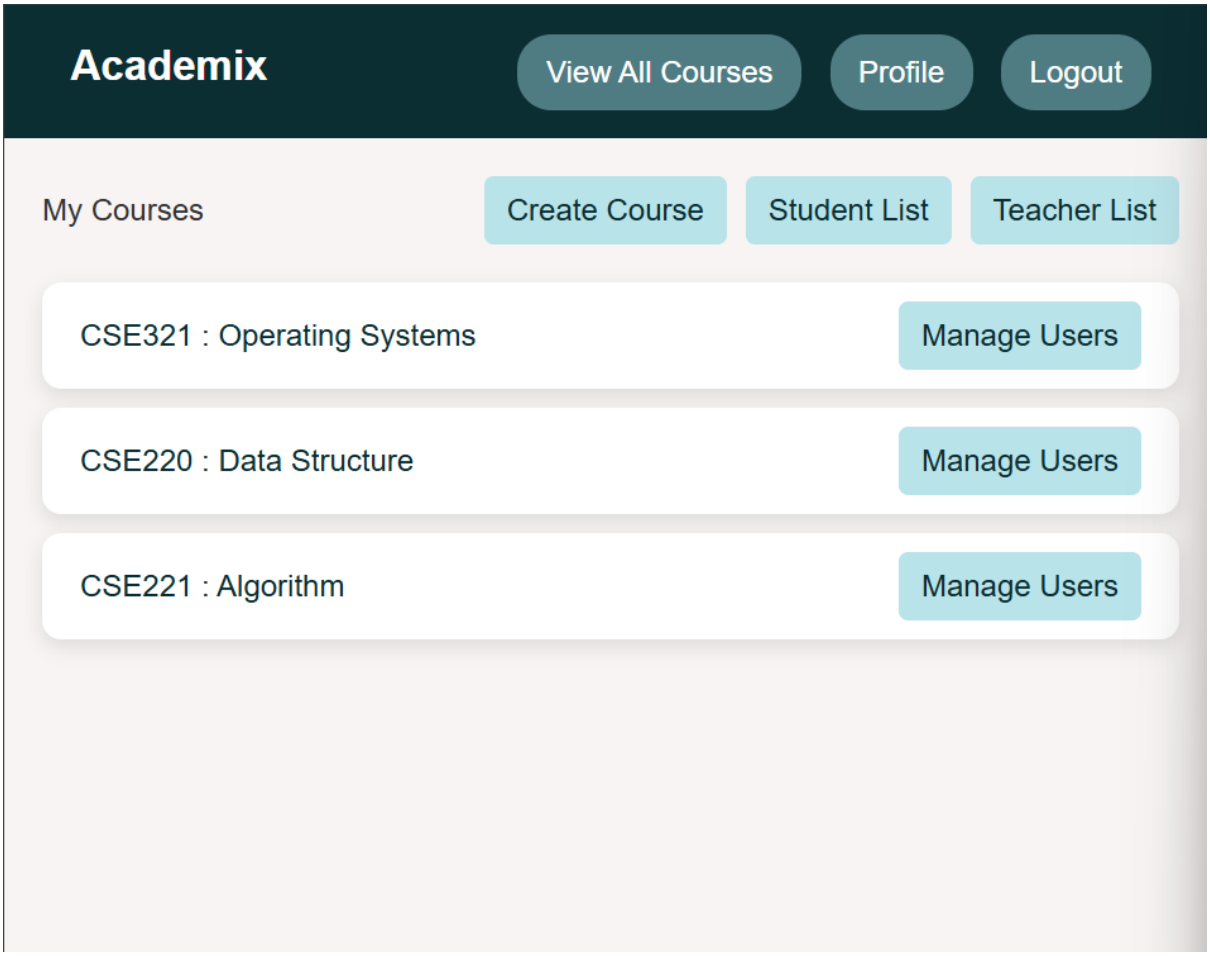
Created Assignments

No assignments created yet.

5. Tecaher can create a course

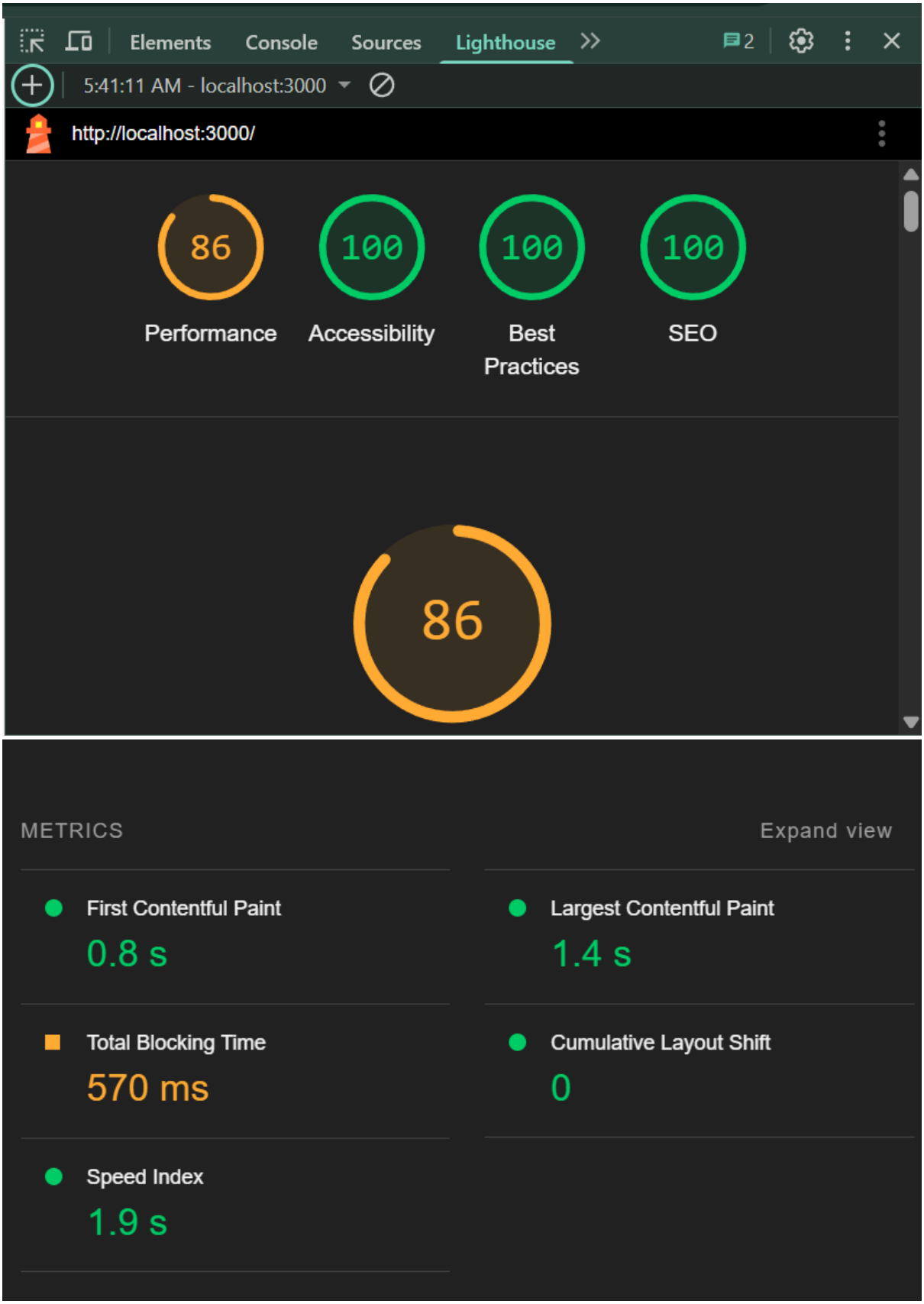


6. If the user is an Admin, he will see this page upon login.



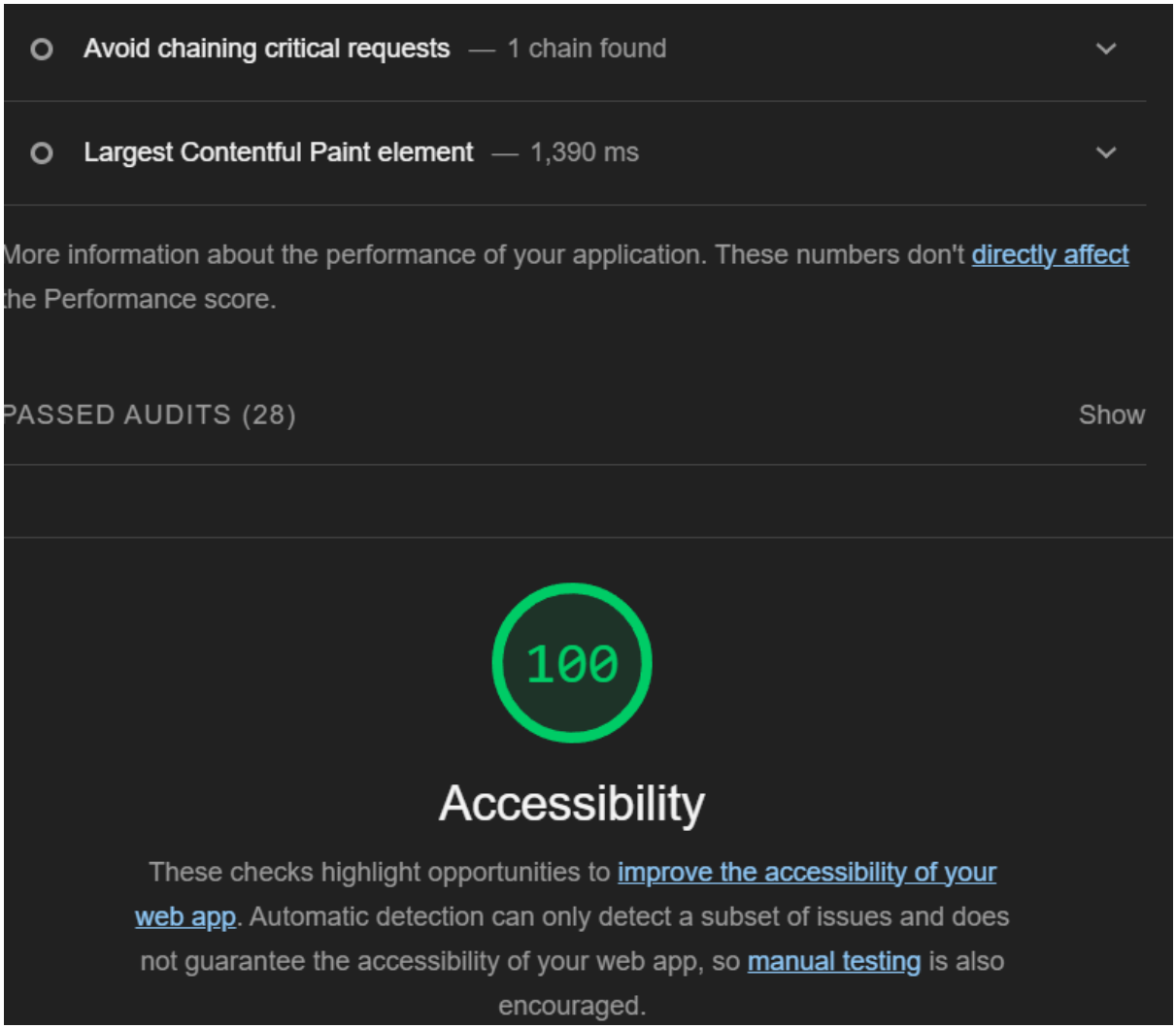
By clicking “Create Course” he can create a course, Student list and teacher list shows the teachers and students of this page.

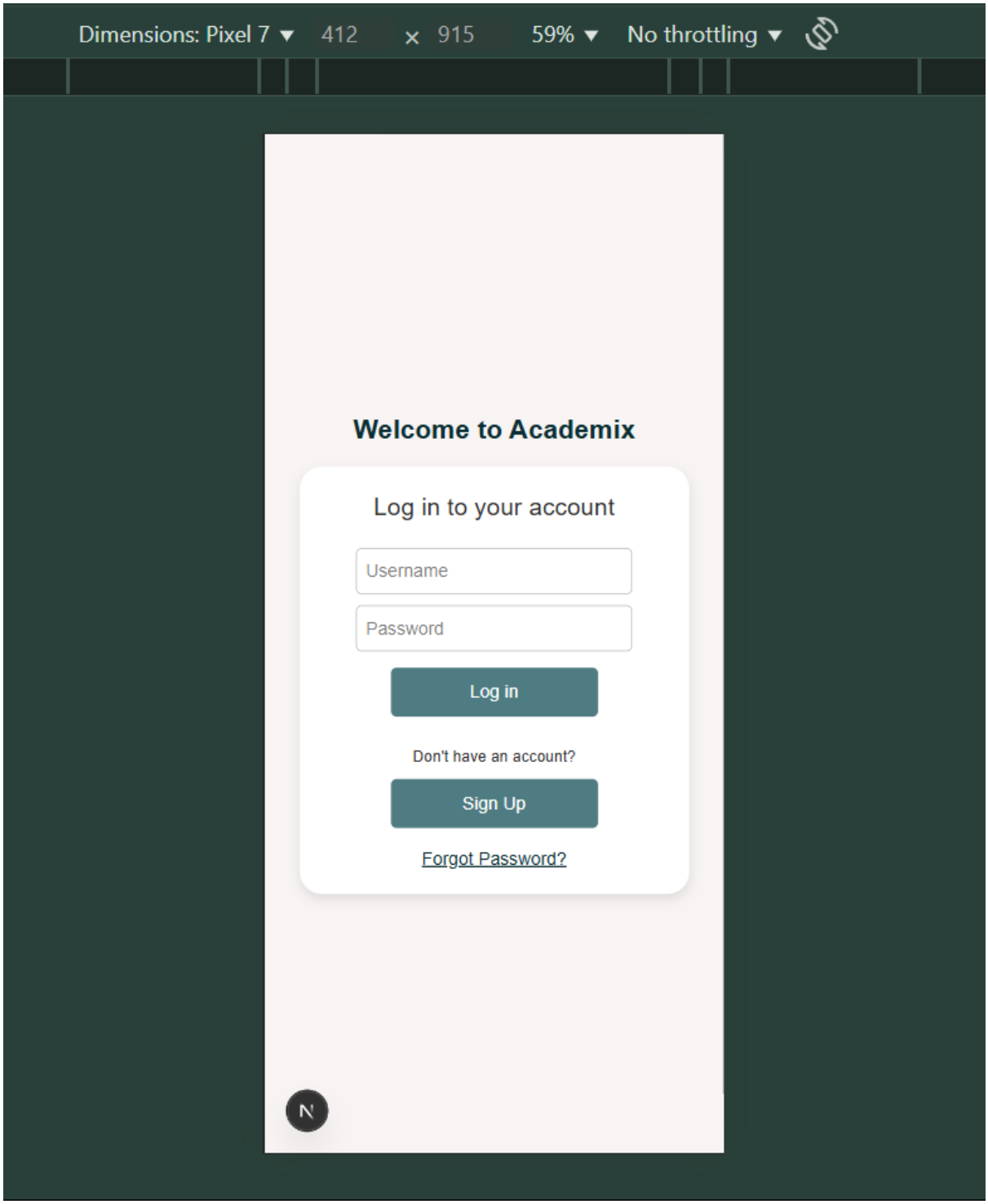
8. Performance and Network Analysis



DIAGNOSTICS

- Minimize main-thread work — 2.3 s
- Page prevented back/forward cache restoration — 3 failure reasons
- Minify JavaScript — Potential savings of 131 KiB
- Eliminate render-blocking resources — Potential savings of 0 ms
- Avoid serving legacy JavaScript to modern browsers — Potential savings of 8 KiB
- Reduce unused JavaScript — Potential savings of 139 KiB
- Avoid long main-thread tasks — 9 long tasks found
- Avoid non-composited animations — 2 animated elements found
- Avoid chaining critical requests — 1 chain found


Avoid chaining critical requests — 1 chain found
Largest Contentful Paint element — 1,390 ms
More information about the performance of your application. These numbers don't directly affect the Performance score.
PASSED AUDITS (28)
Show
100
Accessibility
These checks highlight opportunities to improve the accessibility of your web app. Automatic detection can only detect a subset of issues and does not guarantee the accessibility of your web app, so manual testing is also encouraged.



9. Github Repo [Public]

https://github.com/Arpa4517/CSE471_academix

10. Link of Deployed Project

11. Individual Contribution

Group member - 01	
Name: Ahmed Riasat	Student ID: 24141253
Functional Requirements which are developed by this member:	
1. The whole backend part of this project	

Group member - 02	
Name: Mustafina Joarder	Student ID: 22101215
Functional Requirements which are developed by this member:	
1. The whole frontend part of this project	

12. References

- How to Code : Google's Go(golan) Programming Language
  https://www.udemy.com/course/learn-how-to-code/?couponCode=CP130525
- Go by example
  https://gobyexample.com/
- Implementing JWT authentication in Go
  https://permify.co/post/jwt-authentication-go/
- Gin Web Framework
  https://gin-gonic.com
- Golang official documentation
  https://go.dev/doc/tutorial/web-service-gin
- This project benefited from the assistance of GitHub Copilot, an AI pair programmer
- Traversy Media

https://www.youtube.com/watch?v=LDB4uaJ87e0&t=115s&ab_channel=TraversyMedia

- Codevolution
  https://www.youtube.com/@Codevolution
- Permissions (access control) in web apps

  https://dev.to/wasp/permissions-access-control-in-web-apps-j6b

- PostgreSQL Official Documentation
  https://www.postgresql.org/
- Knowledge 360
  https://www.youtube.com/@Knowledge360Channel/videos